

hw7: the ISO9660 file system

Due date: Mon Nov 27, at noon.

In this assignment, we implement the ISO9660 file system, popularly used on CD-ROM media. A template project is available in the course repository, <https://github.com/bitslab/xv6-public> under the **iso9660fs** branch. Submit your work to github classroom, using [this invitation link](#).

The template adds a virtual file system implementation to xv6, as well as most of the pieces you need in order to implement a rudimentary ISO 9660 file system driver. Your job is to put them together into an actual file system driver, rather than various pieces of functional code.

Wikipedia has a great page on the file system itself. https://en.wikipedia.org/wiki/ISO_9660

For your reference, the template contains a relatively complete implementation of the procfs file system. Some of the ideas in procfs.c will carry over well to the ISO9660 file system.

Step 1: root folder directory listing (25%)

Test the template by mounting an ISO9660 image to your root file system.

```
mkdir mnt
mount iso9660fs cdrom mnt
```

If you list the contents of /mnt, you will find it has some fake files. As a first step, we want to show accurate file names in this listing. Your job is to write to the passed-in buffer a number of directory entries that correspond to what exists on the ISO image.

In the template code, the `mount` command above prints some things to console. This is not typical behavior for `mount` - it is there to give you a start on implementing the directory listing. Among other things, it prints the contents of the root folder on the ISO9660 image. Reading this code, you can see how to read the extent of a directory file, and traverse the directory entries it contains.

Your task, then, is to make the command `"ls mnt"` print out the filenames contained in the root folder of the image (without changing `ls`). `ls` reads the contents of the directory file via repeated uses of the `read` system call, which eventually leads to the `iso9660fs_readi` function in `iso9660fs.c`. When this function is given a directory inode, write appropriate `struct dentry` values to the passed in buffer, and return the number of bytes you wrote.

You may assume that file names are at most 14 bytes, so simply copying over the appropriate bytes will work. Also, file names in ISO9960 are followed by a `;"1"` suffix. This is not a good thing for a bash prompt (which interprets `;` as part of a bash command, not part of a filename). The suffix is typically hidden, and you should do so as well - simply don't copy the `;"1"` part to the `dentry`. The first two entries are for `"."` and `".."`, but they don't have file names. Add the `"."` and `".."` names when listing the directory.

Finally, the `dentry` also needs an inode number, but there are no inodes in ISO9660. Instead, all the metadata about a file is stored within the directory entry itself. Thus, simply return an "inode number" that you can later use to find the directory entry (on disk) that corresponds to this file.

Step 2: accurate inode values (25%)

Once you have completed step 1, you should see a listing of (only) files of size 100, but with different inode numbers. The `iso9660fs_ipopulate` function is responsible for filling in the remaining correct values for an inode that so far has only the `dev` and `inum` values set.

The values that you need to populate for now are the `type` the `size`, so that `ls mnt` shows appropriate types and sizes. The "INDEX.HTML" file is a normal file, while the other three are folders. The screen shot below shows you what to expect from a correct solution. The inode numbers do not need to be these values, anything that lets your `ipopulate` function find the correct directory entry will do.

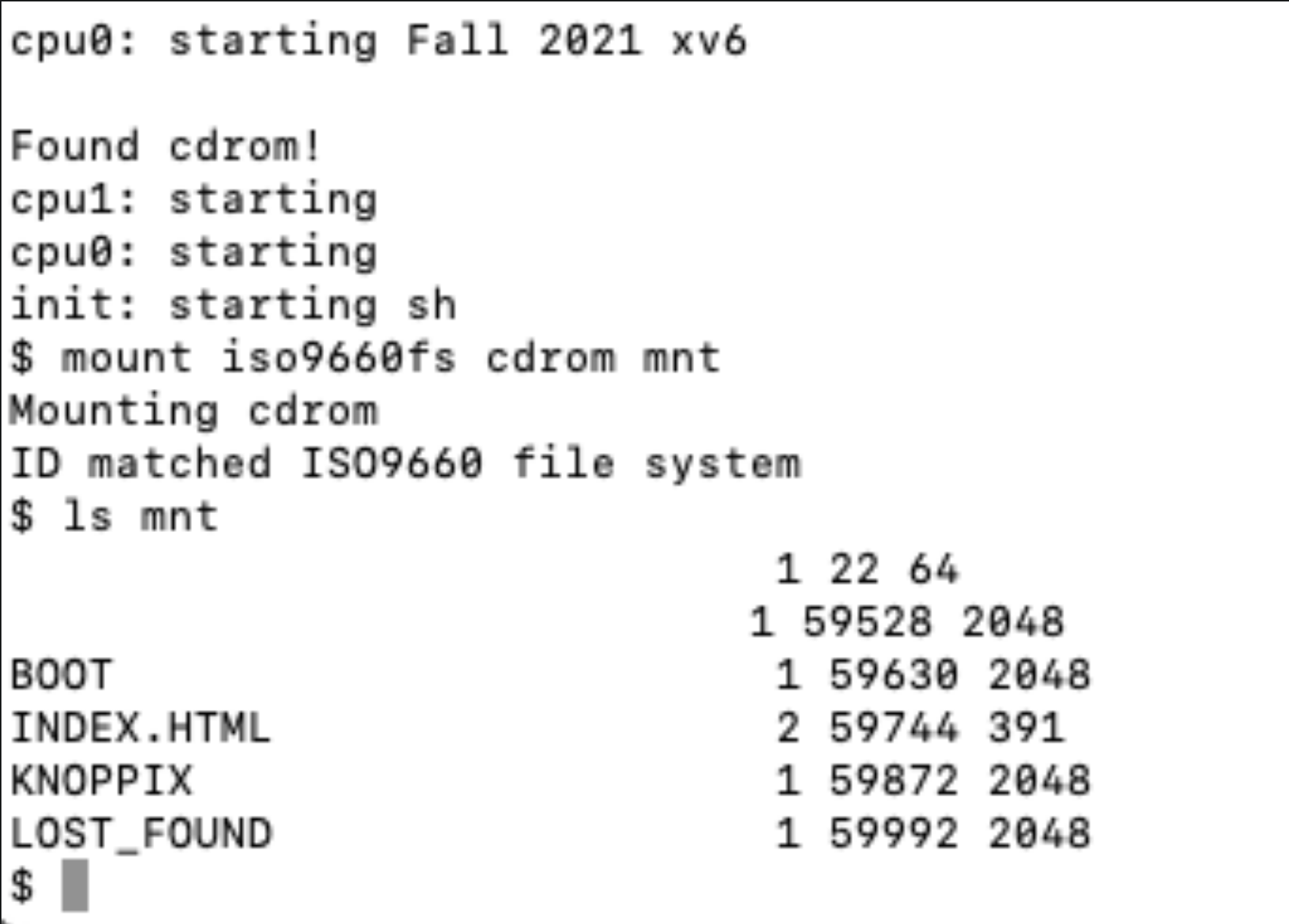


Figure 1: example output showing a directory listing using the `ls` command.

Step 3: file contents (25%)

Using the template code, every file contains only hard-coded "fake" text. In this step, we update `readi` and `ipopulate` to instead support reading the actual contents of the file on disk. In ISO9660, the contents of the file are stored contiguously on disk, starting at the **2048-byte** block indicated by the `extent` field in the (`struct iso9660_dir_s`) directory entry.

Thus, update `ipopulate` to also remember the extent in the `struct inode` (I used the first element in the `addrs[]` array for this), then change `readi` to have it return the appropriate contents from the file on disk.

NOTE: Pay attention to the offset parameter to the readi function! This tells you which bytes of the file the program wants to see next.

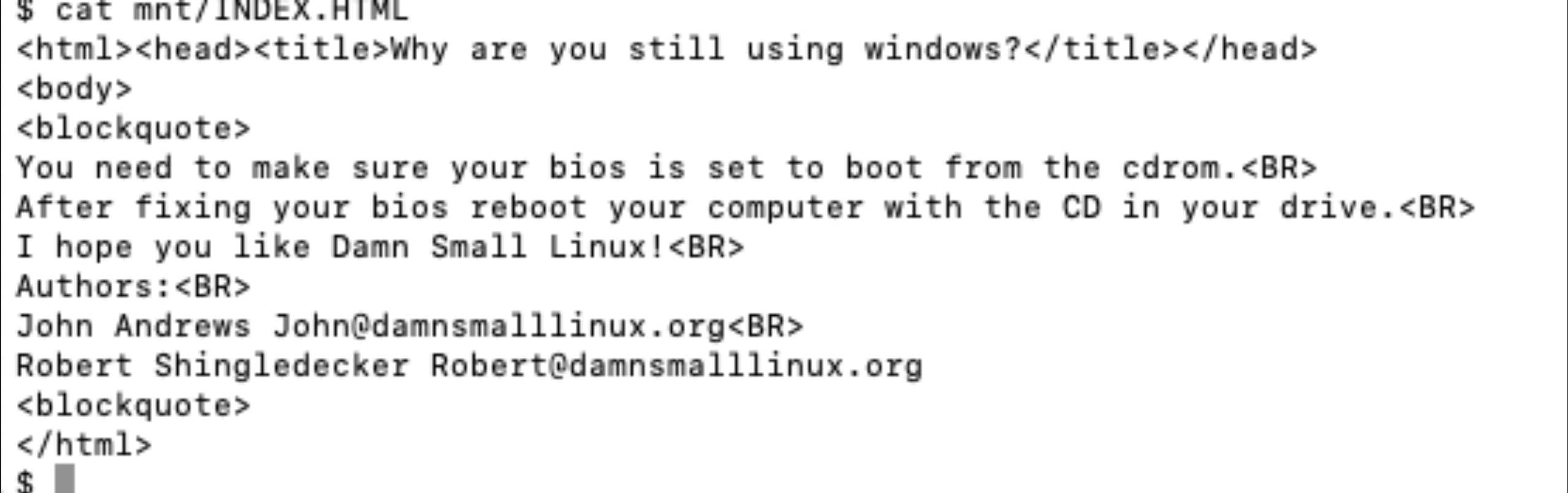


Figure 2: Example output reading the contents of the file INDEX.HTML

Step 4: list and access contents of other directories (25%)

So far, we support only listing the contents of the root folder, and accessing the contents of the one file within that folder. Now, extend `readi` to support listing the contents of other folders. If you do it right, this should also automatically give you access to file contents in other folders.

Previously, we returned the contents of the root directory no matter which directory inode was passed in to `readi`. Update `readi` to pay attention to which directory the user wants to read. For all directories except the root directory, this works analogous to file contents - use the extent stored in the `struct inode` to find the directory contents, and return `struct dentry` values corresponding to the offset requested by the user.

For the root directory, things are a little different. Its inode (the one corresponding to `mnt`, in the examples above) is actually stored on the main disk, not in the ISO9660 disk image. Thus, the `struct inode` contents came from the `xv6` file system, and the `inum` has no ISO9660 meaning. For this special folder, we instead need to find the root folder contents using the primary volume descriptor (as in step 1).

To distinguish between the mount point and other folders, use the `"mounted_dev"` value in the `struct inode`. If `mounted_dev` is non-null, this is a mount point, and you should return the contents of the root folder contents for the device, instead of the contents of the mount point on the parent disk.

Note: this should work for using any directory as mount point- don't hard code anything to work only for a folder "mnt".

HINT: when reading the BOOT.MSG file below, you're likely to run into an "incorrect blockno" panic. Read the code that issues the panic, and work out what happened, and what to do about it.

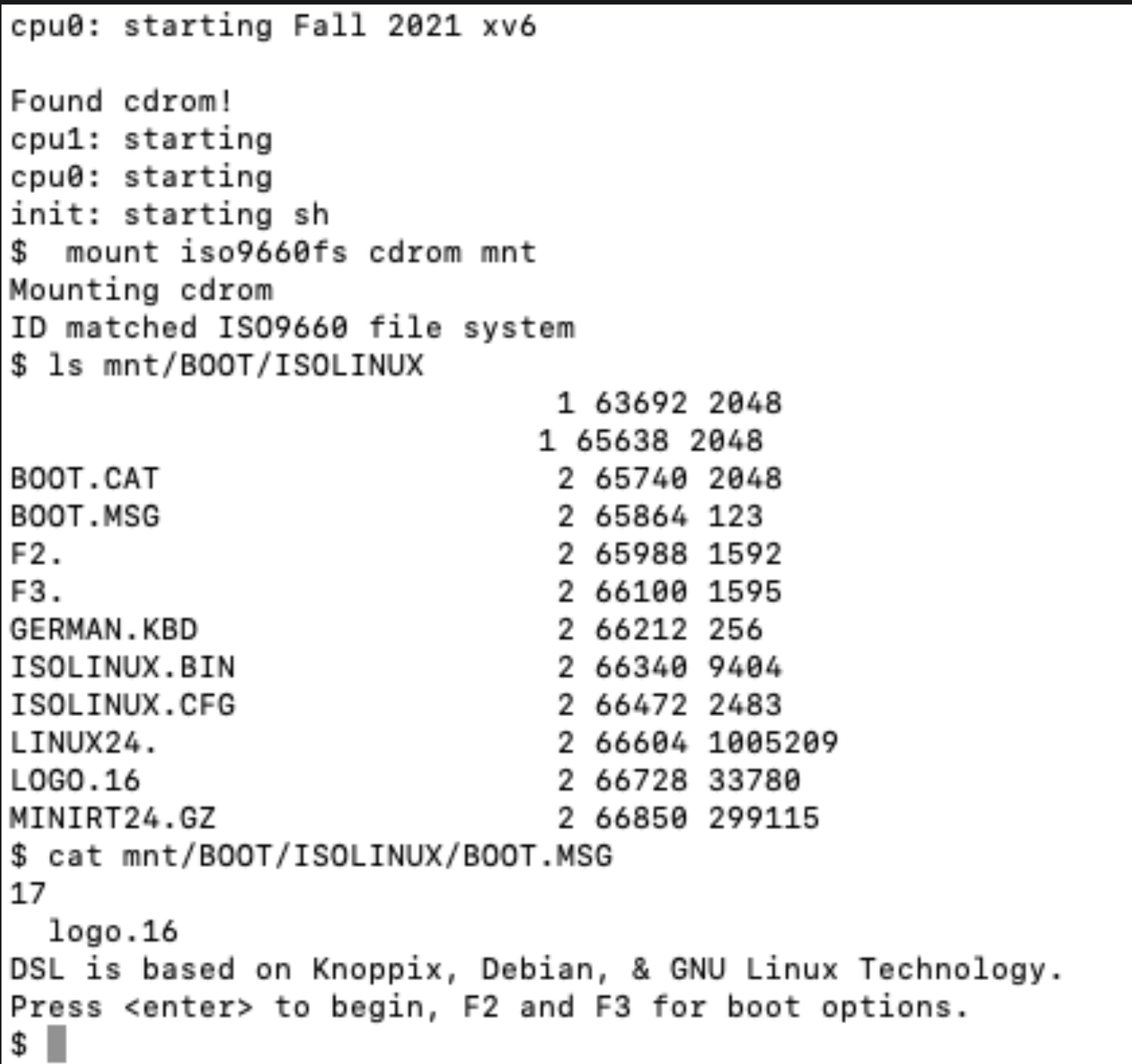


Figure 3: example output using other directories - showing both directory contents, and contents of a file in a nested directory.